LMCache

为所有人构建开放的统一 KV Cache 管理层

LMCache：面向多引擎、多硬件协作的开源 KV Cache 管理层

毛宝龙 | 郑春晓

关于讲者



毛宝龙

LMCache Maintainer & 核心架构师
(PyTorch Ecosystem)

- 项目第一贡献者，累计合并 300+
- 底层架构与开源协作
- vLLM Contributor
- Mooncake / Apache Ozone Committer
- 深耕开源 & AI Infra与高性能存缓系统
- 曾任职于腾讯、阿里云、京东

聚焦：Make LMCache to be extensible and pluggable
最近：Multi-tier Caching & Multi Device platform integration



郑春晓

LMCache Maintainer & 核心架构师
(PyTorch Ecosystem)

- 项目第五大贡献者，累计合并 100+ PR
- LMCache 核心 Maintainer
- vLLM Contributor
- 毕业以来专注于大数据存储和KV Cache相关存储
- 曾任职于腾讯任职

主导: DeepSeek MLA、DSA等attention的适配，
专注: 多层级缓存的实现和优化、SGLang 集成

关于讲者



毛宝龙

LMCache Maintainer & 核心架构师
(PyTorch Ecosystem)

- 项目第一贡献者，累计合并 300+
- 底层架构与开源协作
- vLLM Contributor
- Mooncake / Apache Ozone Committer
- 深耕开源 & AI Infra与高性能存缓系统
- 曾任职于腾讯、阿里云、京东

聚焦：Make LMCache to be extensible and pluggable
最近：Multi-tier Caching & Multi Device platform integration



郑春晓

LMCache Maintainer & 核心架构师
(PyTorch Ecosystem)

- 项目第五大贡献者，累计合并 100+ PR
- LMCache 核心 Maintainer
- vLLM Contributor
- 毕业以来专注于大数据存储和KV Cache相关存储
- 曾任职于腾讯任职

主导: DeepSeek MLA、DSA等attention的适配，
专注: 多层级缓存的实现和优化、SGLang 集成

六章内容

01 LMCache 的架构演进

02 LMCache 的可插拔扩展

03 LMCache 开源社区

06 展望和Roadmap

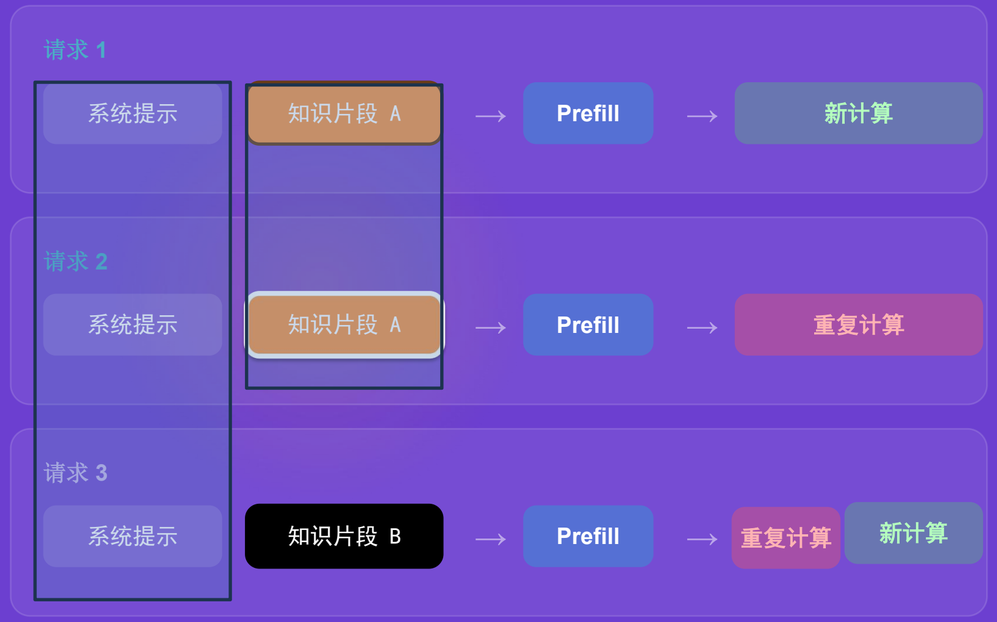
05 使用场景与最佳实践

04 部署、可观测性与容错

01 LMCache 的架构演进

重复前缀让 KV Cache 具备复用价值

没有复用：相同前缀被反复 Prefill



系统提示、RAG 上下文、Few-shot 示例……这些前缀在大量请求中**完全相同**，却被一次次重新计算。

KV Cache 复用：把浪费变成收益

01 减少重复 Prefill 计算

相同前缀直接命中 KV Cache，跳过 Prefill 阶段，**首 Token 延迟显著降低**，用户体验更流畅。

02 通过 Offload 释放 GPU 显存空间

KV Cache 可 Offload 到 CPU / L2 存储，**GPU 显存可承载更长上下文或更高并发**，单卡利用率最大化。

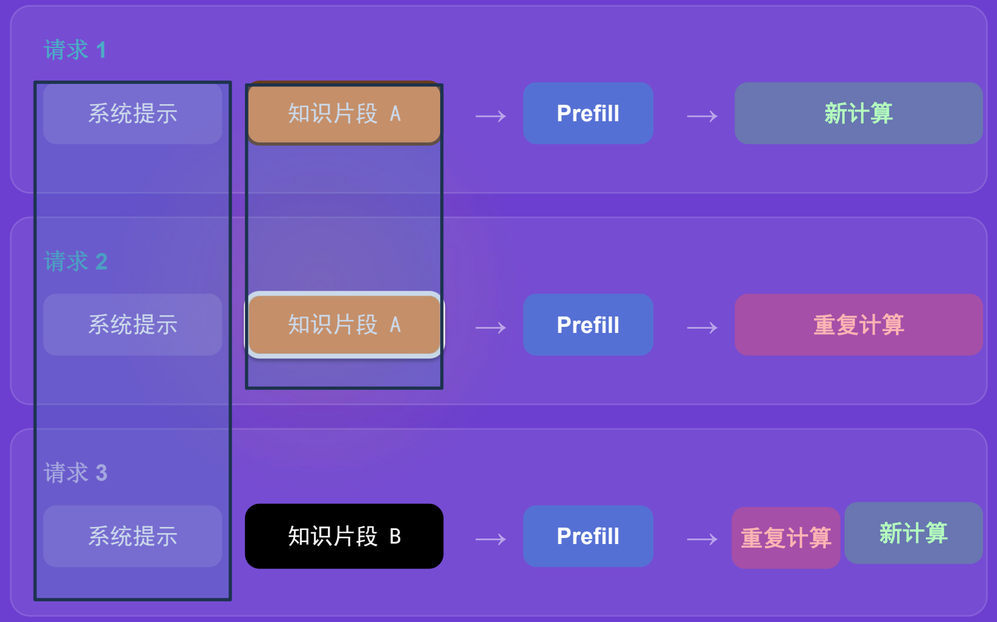
03 跨实例 / 跨节点共享

一份 KV Cache 可被多个推理实例复用，**集群整体吞吐大幅提升**，规模化部署成本显著下降。

核心结论：同样的前缀不用反复算——KV Cache 存一次、用多次，**省算力、省显存、提吞吐**。

重复前缀让 KV Cache 具备复用价值

没有复用：相同前缀被反复 Prefill



系统提示、RAG 上下文、Few-shot 示例……这些前缀在大量请求中**完全相同**，却被一次次重新计算。

KV Cache 复用：把浪费变成收益

01 减少重复 Prefill 计算

相同前缀直接命中 KV Cache，跳过 Prefill 阶段，**首 Token 延迟显著降低**，用户体验更流畅。

02 通过 Offload 释放 GPU 显存空间

KV Cache 可 Offload 到 CPU / L2 存储，**GPU 显存可承载更长上下文或更高并发**，单卡利用率最大化。

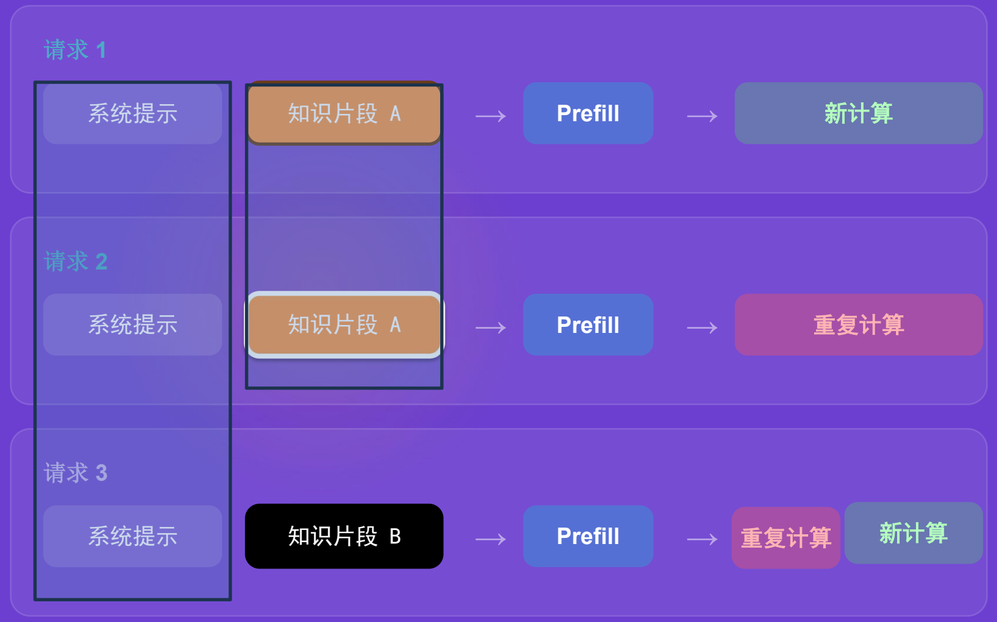
03 跨实例 / 跨节点共享

一份 KV Cache 可被多个推理实例复用，**集群整体吞吐大幅提升**，规模化部署成本显著下降。

核心结论：同样的前缀不用反复算——KV Cache 存一次、用多次，**省算力、省显存、提吞吐**。

重复前缀让 KV Cache 具备复用价值

没有复用：相同前缀被反复 Prefill



系统提示、RAG 上下文、Few-shot 示例……这些前缀在大量请求中**完全相同**，却被一次次重新计算。

KV Cache 复用：把浪费变成收益

01 减少重复 Prefill 计算

相同前缀直接命中 KV Cache，跳过 Prefill 阶段，**首 Token 延迟显著降低**，用户体验更流畅。

02 通过 Offload 释放 GPU 显存空间

KV Cache 可 Offload 到 CPU / L2 存储，**GPU 显存可承载更长上下文或更高并发**，单卡利用率最大化。

03 跨实例 / 跨节点共享

一份 KV Cache 可被多个推理实例复用，**集群整体吞吐大幅提升**，规模化部署成本显著下降。

核心结论：同样的前缀不用反复算——KV Cache 存一次、用多次，**省算力、省显存、提吞吐**。

LMCache 把 KV Cache 变成有生命周期的可被管理资源



LMCache 的价值：不只是“快一点”，而是让 KV Cache 真正进入基础设施的管理范围。

LMCache 把 KV Cache 变成有生命周期的可被管理资源



LMCache 的价值：不只是“快一点”，而是让 KV Cache 真正进入基础设施的管理范围。

LMCache 的演进历程：从 vLLM 内部实现到独立服务

in-tree 原型 -> 外部 Connector 优先 -> 独立 MP Server

2024.12 未合并



@ApostaCyihua

vLLM #10874

CPU Offloading 原型

in-tree 路径验证
后续方案的早期雏形

in-tree → LMCache v0

2025.2 已合并



@YaoJiayi

vLLM #12953

LMCache v0 kv connector

in-tree connector
Out-tree lmcache

in-tree → LMCache v0

2025.04 已合并



@ApostaCyihua

vLLM #15960

KV Connector V1 API

Scheduler / Worker
Connector 职责成形

稳定接口层

2025.04 已合并



@ApostaCyihua

vLLM #16625

LMCache 接入 vLLM v1

Offload · 共享 · P/D
接入统一 Connector

LMCache v1

2026.04 已合并



@maobaolong

vLLM #38301

外部 Connector 优先

kv_connector_module_path
可覆盖同名内置 Connector

vLLM 允许 out-of-tree 自维护

2026.05 已合并



@chunxiaozheng

LMCache #3235

MP Connector 归 LMCache

最新 MP Connector
在 LMCache 仓库侧维护

独立 Connector 发布节奏

现在

LMCache

LMCache MP mode

out-of-tree library
+ 独立 LMCache Server
多推理引擎，多存储，多硬件。
通用的统一 KVCache 管理层

接口稳定 · 实现外置 · 服务解耦

LMCache 的演进历程：从 vLLM 内部实现到独立服务

in-tree 原型 -> 外部 Connector 优先 -> 独立 MP Server

2024.12 未合并



@ApostaCyihua

vLLM #10874

CPU Offloading 原型

in-tree 路径验证
后续方案的早期雏形

in-tree → LMCache v0

2025.2 已合并



@YaoJiayi

vLLM #12953

LMCache v0 kv connector

in-tree connector
Out-tree lmcache

in-tree → LMCache v0

2025.04 已合并



@ApostaCyihua

vLLM #15960

KV Connector V1 API

Scheduler / Worker
Connector 职责成形

稳定接口层

2025.04 已合并



@ApostaCyihua

vLLM #16625

LMCache 接入 vLLM v1

Offload · 共享 · P/D
接入统一 Connector

LMCache v1

2026.04 已合并



@maobaolong

vLLM #38301

外部 Connector 优先

kv_connector_module_path
可覆盖同名内置 Connector

vLLM 允许 out-of-tree 自维护

2026.05 已合并



@chunxiaozheng

LMCache #3235

MP Connector 归 LMCache

最新 MP Connector
在 LMCache 仓库侧维护

独立 Connector 发布节奏

现在

LMCache

LMCache MP mode

out-of-tree library
+ 独立 LMCache Server
多推理引擎，多存储，多硬件。
通用的统一 KVCache 管理层

接口稳定 · 实现外置 · 服务解耦

LMCache 的演进历程：从 vLLM 内部实现到独立服务

in-tree 原型 -> 外部 Connector 优先 -> 独立 MP Server

2024.12 未合并



@ApostaCyihua

vLLM #10874

CPU Offloading 原型

in-tree 路径验证
后续方案的早期雏形

in-tree → LMCache v0

2025.2 已合并



@YaoJiayi

vLLM #12953

LMCache v0 kv connector

in-tree connector
Out-tree lmcache

in-tree → LMCache v0

2025.04 已合并



@ApostaCyihua

vLLM #15960

KV Connector V1 API

Scheduler / Worker
Connector 职责成形

稳定接口层

2025.04 已合并



@ApostaCyihua

vLLM #16625

LMCache 接入 vLLM v1

Offload · 共享 · P/D
接入统一 Connector

LMCache v1

2026.04 已合并



@maobaolong

vLLM #38301

外部 Connector 优先

kv_connector_module_path
可覆盖同名内置 Connector

vLLM 允许 out-of-tree 自维护

2026.05 已合并



@chunxiaozheng

LMCache #3235

MP Connector 归 LMCache

最新 MP Connector
在 LMCache 仓库侧维护

独立 Connector 发布节奏

现在

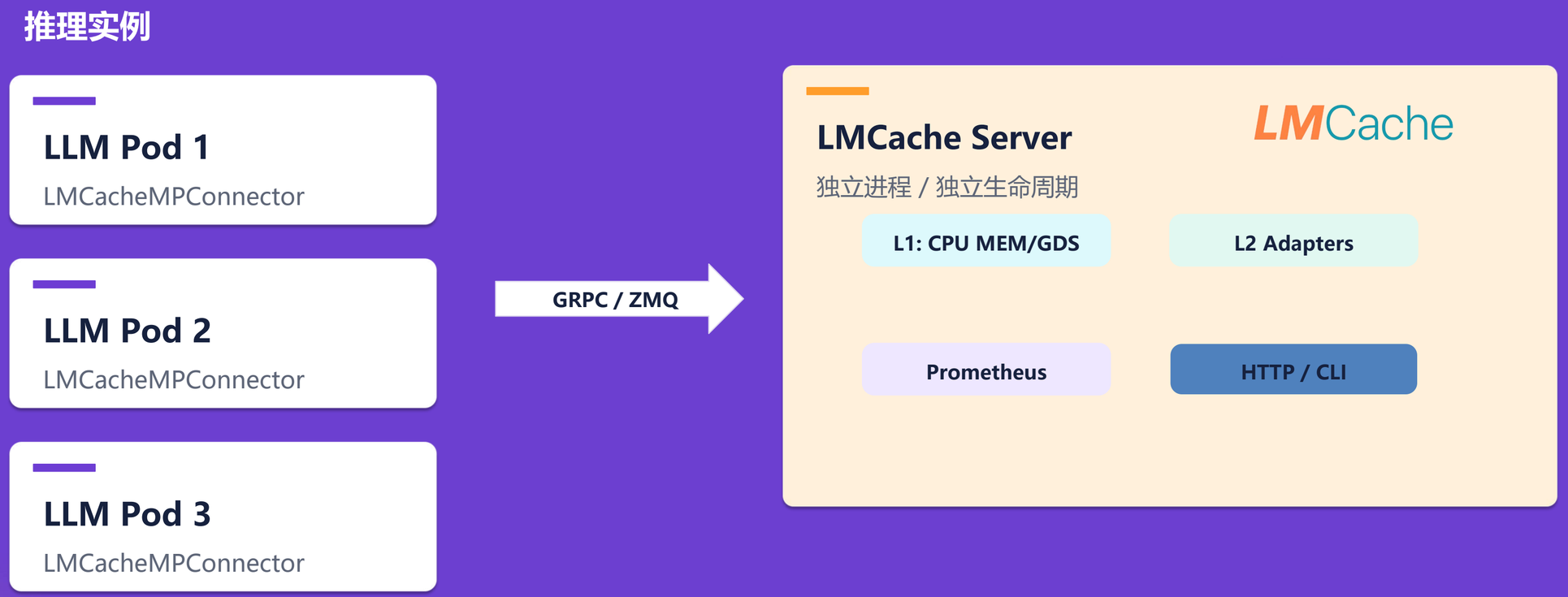
LMCache

LMCache MP mode

out-of-tree library
+ 独立 LMCache Server
多推理引擎，多存储，多硬件。
通用的统一 KVCache 管理层

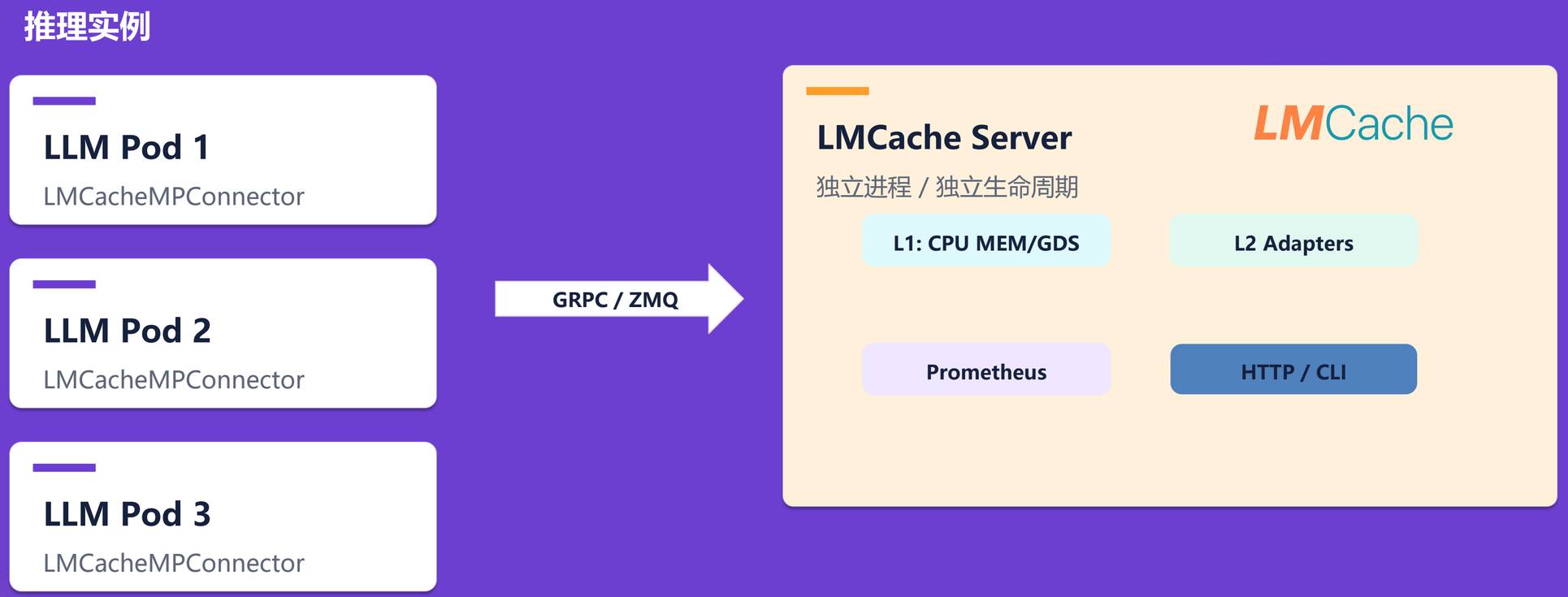
接口稳定 · 实现外置 · 服务解耦

LMCache MP mode 把计算与 KV Cache 服务解耦



共享缓存、进程隔离、独立容量规划，是 MP mode 的主要价值

LMCache MP mode 把计算与 KV Cache 服务解耦



共享缓存、进程隔离、独立容量规划，是 MP mode 的主要价值

02 LMCache 的可插拔扩展

Serving engine 接入与模型可扩展

推理引擎 Connector

- **vLLM** — KV Connector
- **SGLang** — KV Connector
- **TensorRT-LLM** — KV Connector
- **AMD-ATOM** — KV Connector

LMCache

统一 KV Cache 管理层

Connector 生命周期
EngineKVFormat · KV groups

主流新模型布局

- **DeepSeek V4** — 正确性测试与持续适配
- **Kimi K3** — Day-0 recipe + MLA 修复
- **MiniMax M3** — 混合 KV group 适配

分层设计，Serving engine变化主要影响 connector，模型变化影响 EngineKVFormat 和 KV group

Serving engine 接入与模型可扩展

推理引擎 Connector

- **vLLM** — KV Connector
- **SGLang** — KV Connector
- **TensorRT-LLM** — KV Connector
- **AMD-ATOM** — KV Connector

LMCache

统一 KV Cache 管理层

Connector 生命周期
EngineKVFormat · KV groups

主流新模型布局

- **DeepSeek V4** — 正确性测试与持续适配
- **Kimi K3** — Day-0 recipe + MLA 修复
- **MiniMax M3** — 混合 KV group 适配

分层设计，Serving engine变化主要影响 connector，模型变化影响 EngineKVFormat 和 KV group

LMCache 缓存多层级 L1，L2 可以独立扩展




```
lmcache server --l2-adapter '{"type":"plugin","module_path":"my_plugin.l2","class_name":"MyL2Adapter"}'
```

L2 Adapter 可以作为独立 pip 包 out-of-tree 维护，不需要修改 LMCache Core

LMCache 缓存多层级 L1，L2 可以独立扩展




```
lmcache server --l2-adapter '{"type":"plugin","module_path":"my_plugin.l2","class_name":"MyL2Adapter"}'
```

L2 Adapter 可以作为独立 pip 包 out-of-tree 维护，不需要修改 LMCache Core

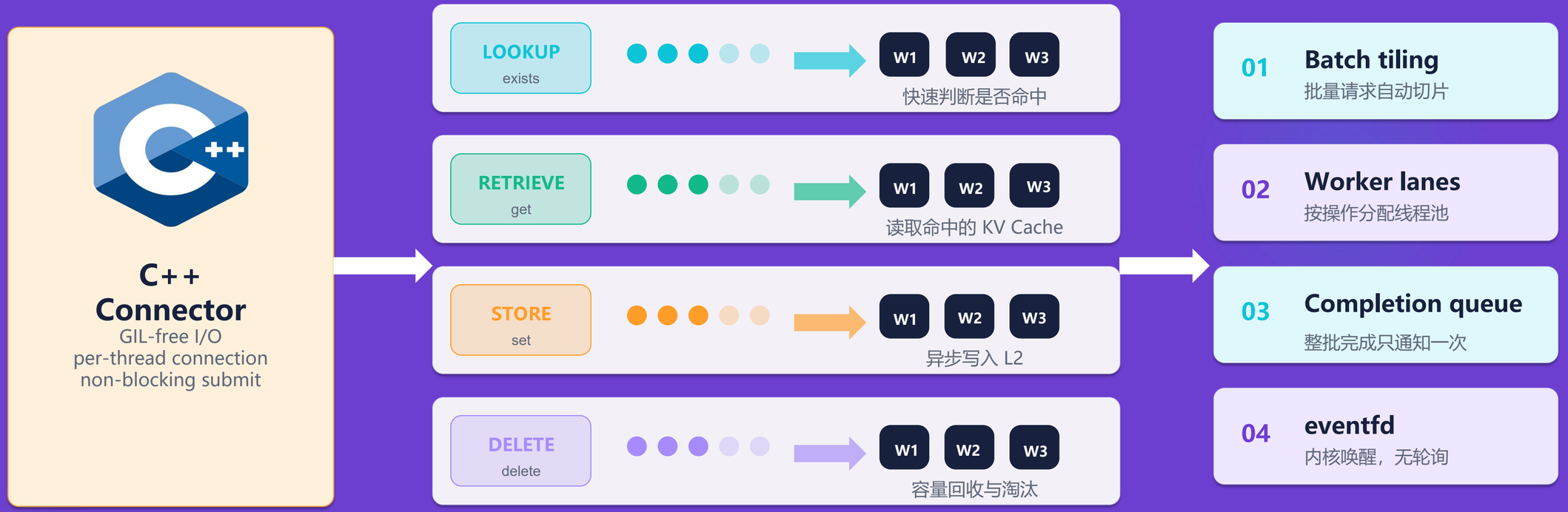
LMCache 缓存多层级 L1，L2 可以独立扩展




```
lmcache server --l2-adapter '{"type":"plugin","module_path":"my_plugin.l2","class_name":"MyL2Adapter"}'
```

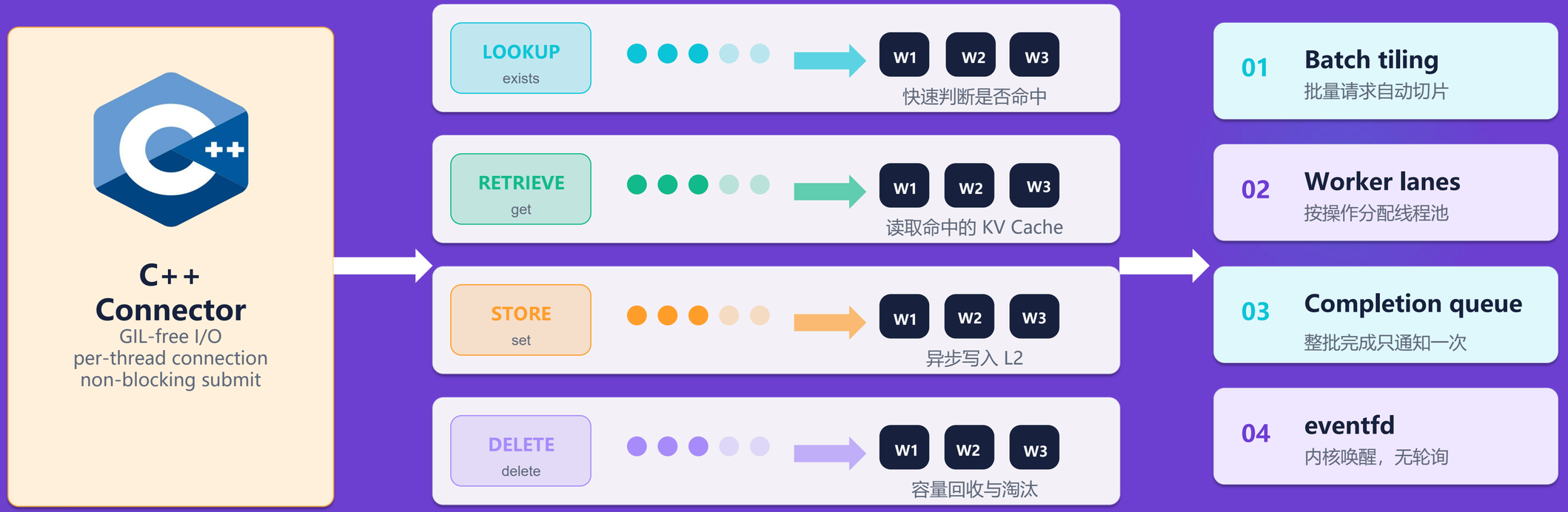
L2 Adapter 可以作为独立 pip 包 out-of-tree 维护，不需要修改 LMCache Core

Native Adapter 用独立队列隔离不同操作



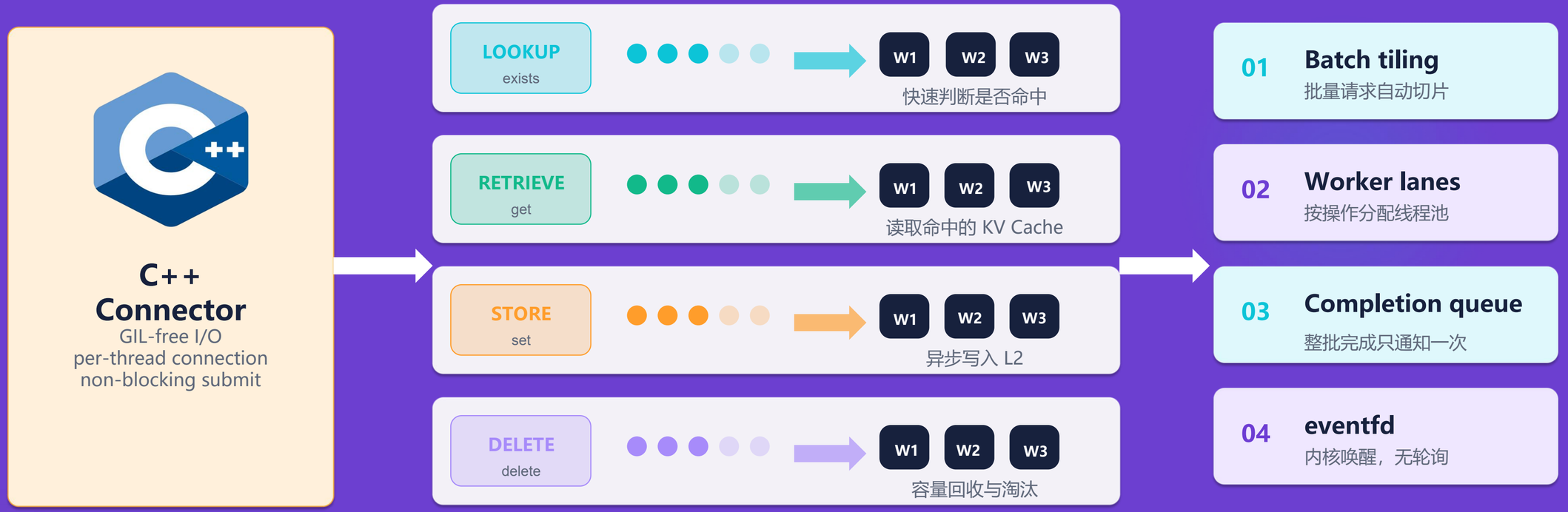
per_op_workers 可以分别配置 lookup / retrieve / store / delete 的并发，避免慢写入阻塞命中判断

Native Adapter 用独立队列隔离不同操作



per_op_workers 可以分别配置 lookup / retrieve / store / delete 的并发，避免慢写入阻塞命中判断

Native Adapter 用独立队列隔离不同操作



per_op_workers 可以分别配置 lookup / retrieve / store / delete 的并发，避免慢写入阻塞命中判断

LMCache 多 Platform 可扩展

推理引擎

vLLM

SGL

TensorRT-LLM

AMD-ATOM

LMCache

LMCache Core

统一缓存生命周期

Platform Plugin Boundary

DeviceSpec
总管设备、内存与布局能力

DeviceOps
分配、拷贝与同步算子

CacheContext
Stream context, tmp buffer 管理分配

IPC / Transfer
选择可用数据路径

可扩展 Device Platform

- NVIDIA CUDA
- AMD ROCm
- Intel XPU
- Huawei Ascend
- Moore Threads MUSA
- METAX MACA
- Rebellions NPU
- Device Plugin

Thirt party Device plugin

新增硬件通过 Device Platform 扩展，可 out-tree 可插拔维护

LMCache 多 Platform 可扩展

推理引擎

vLLM

SGL

TensorRT-LLM

AMD-ATOM

LMCache

LMCache Core

统一缓存生命周期

Platform Plugin Boundary

DeviceSpec
总管设备、内存与布局能力

DeviceOps
分配、拷贝与同步算子

CacheContext
Stream context, tmp buffer 管理分配

IPC / Transfer
选择可用数据路径

可扩展 Device Platform

- NVIDIA CUDA
- AMD ROCm
- Intel XPU
- Huawei Ascend
- Moore Threads MUSA
- METAX MACA
- Rebellions NPU
- Device Plugin

Thirt party Device plugin

新增硬件通过 Device Platform 扩展，可 out-tree 可插拔维护

LMCache 多 Platform 可扩展

推理引擎

vLLM

SGL

TensorRT-LLM

AMD-ATOM

LMCache

LMCache Core

统一缓存生命周期

Platform Plugin Boundary

DeviceSpec
总管设备、内存与布局能力

DeviceOps
分配、拷贝与同步算子

CacheContext
Stream context, tmp buffer 管理分配

IPC / Transfer
选择可用数据路径

可扩展 Device Platform

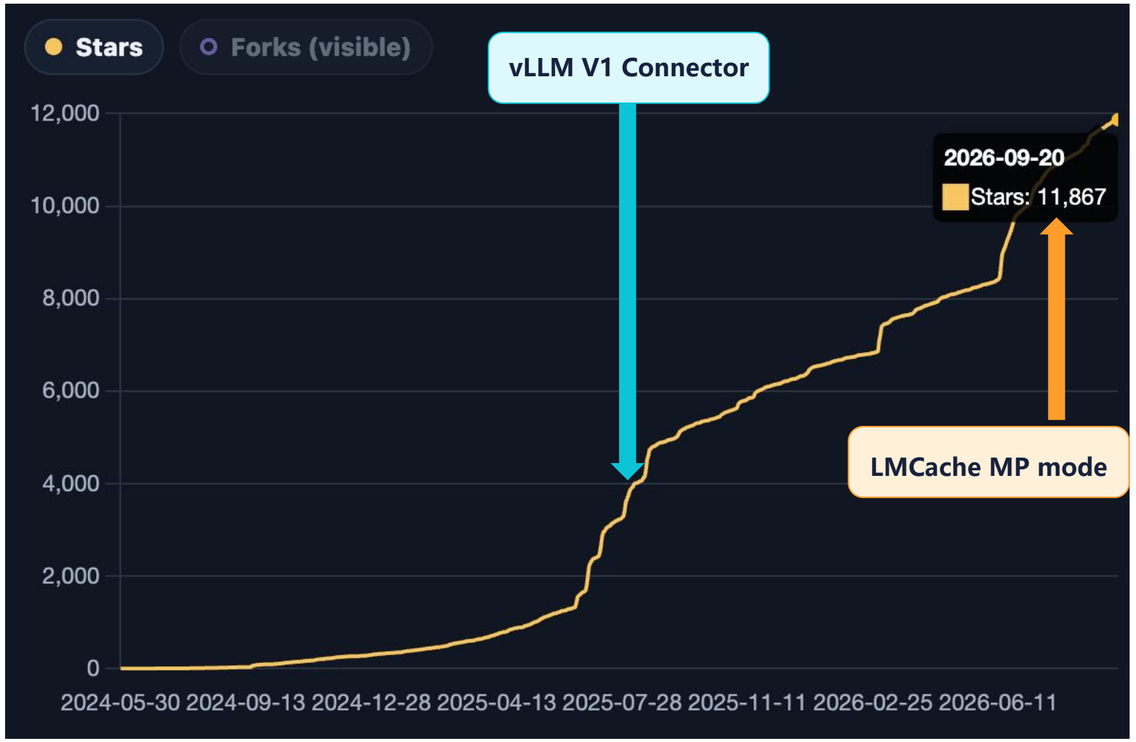
- NVIDIA CUDA
- AMD ROCm
- Intel XPU
- Huawei Ascend
- Moore Threads MUSA
- METAX MACA
- Rebellions NPU
- Device Plugin

Thirt party Device plugin

新增硬件通过 Device Platform 扩展，可 out-tree 可插拔维护

03 LMCache 活跃的开源社区

LMCache 开源生态进入加速增长阶段



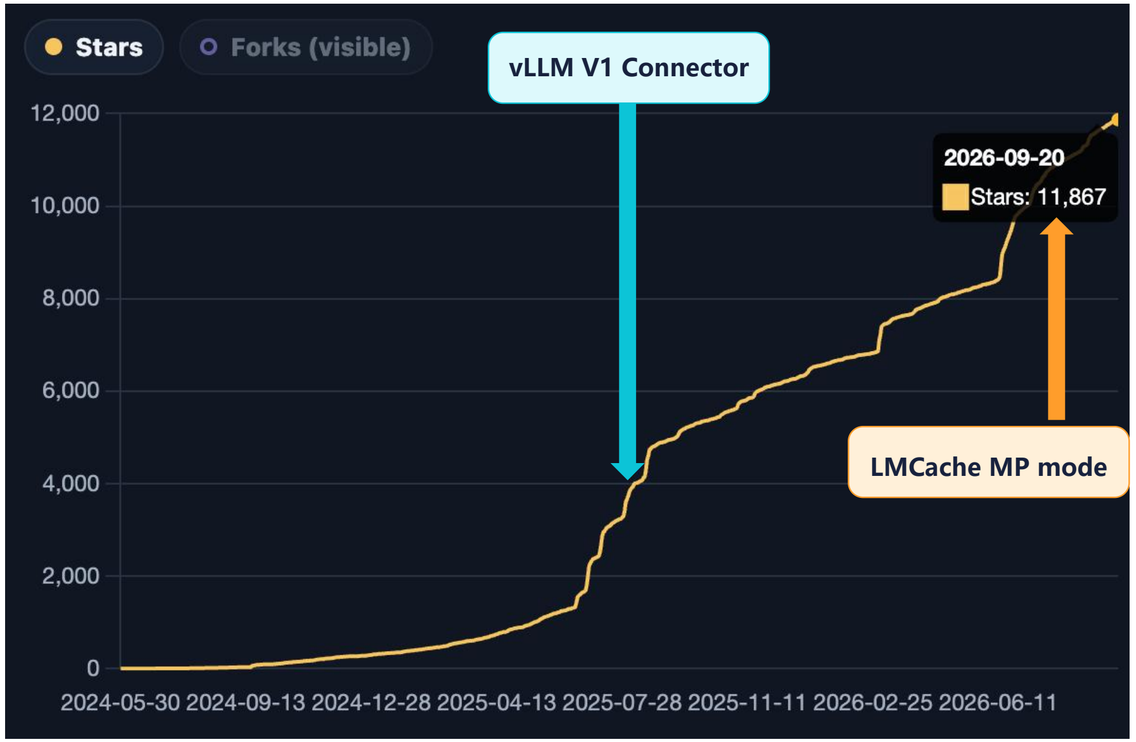
12K+
GitHub Stars
社区关注

~ 400
Contributors
参与共建

1M+
PyPI Downloads
累计下载

github.com/LMCache/LMCache

LMCache 开源生态进入加速增长阶段



12K+
GitHub Stars
社区关注

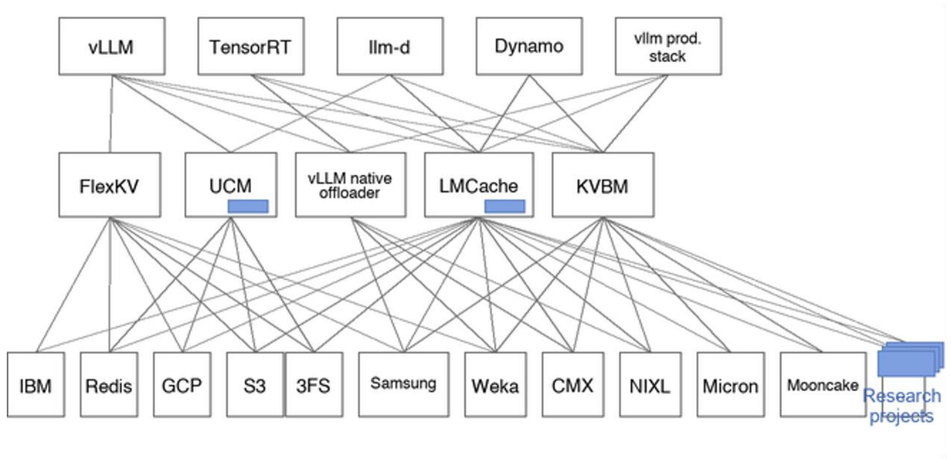
~ 400
Contributors
参与共建

1M+
PyPI Downloads
累计下载

github.com/LMCache/LMCache

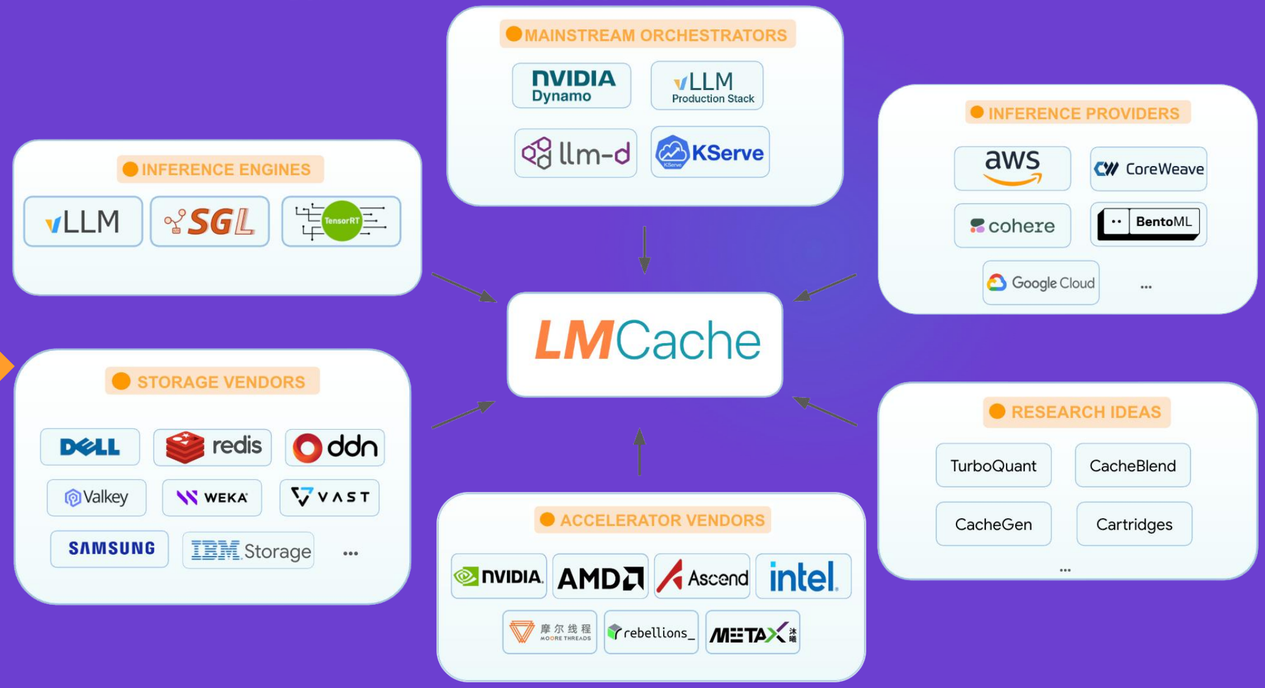
统一缓存层把两两适配收敛为标准边界

没有统一层：N × M 两两适配



每增加一个引擎或后端，都可能引入一批新组合

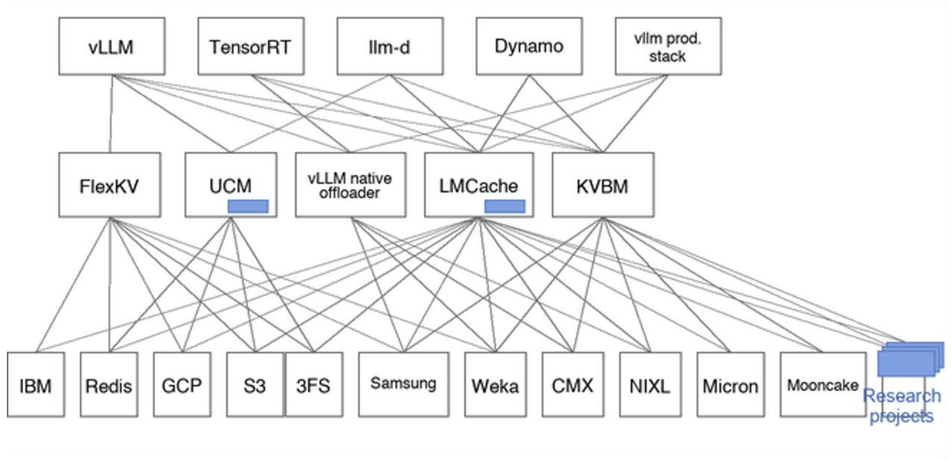
引入 LMCache：引擎和后端各接一次



统一边界减少组合爆炸，让引擎、存储和硬件按各自节奏演进

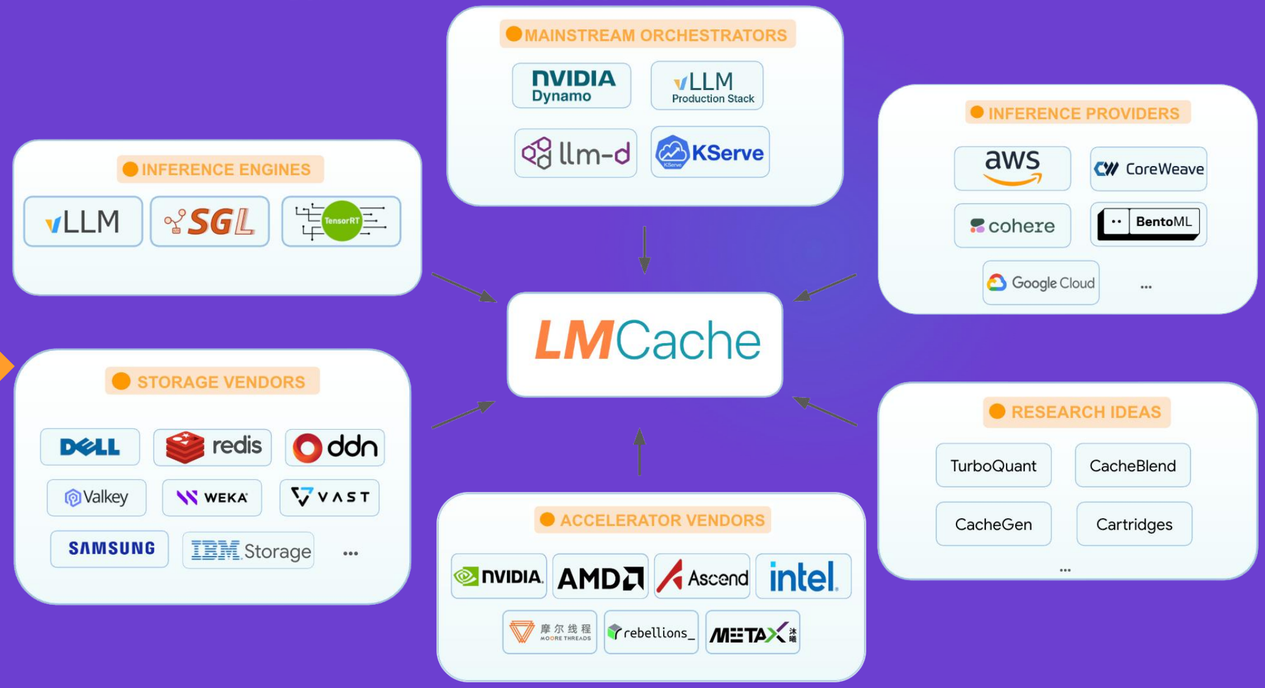
统一缓存层把两两适配收敛为标准边界

没有统一层：N × M 两两适配



每增加一个引擎或后端，都可能引入一批新组合

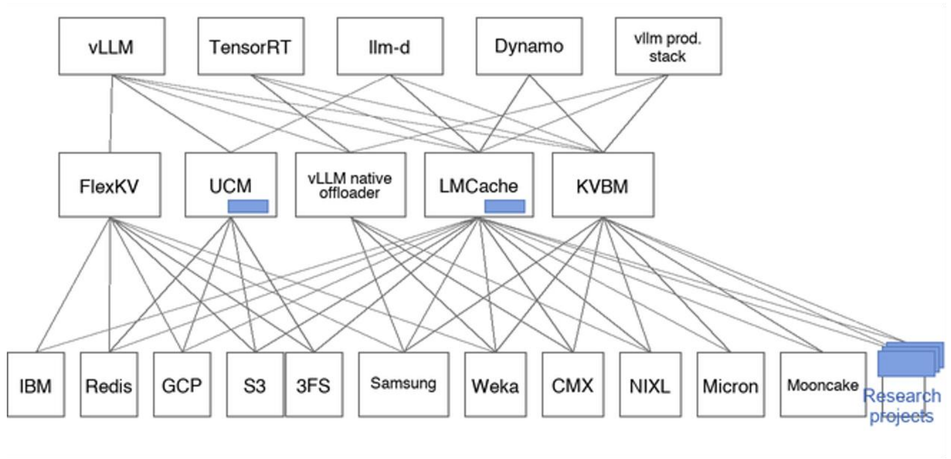
引入 LMCache：引擎和后端各接一次



统一边界减少组合爆炸，让引擎、存储和硬件按各自节奏演进

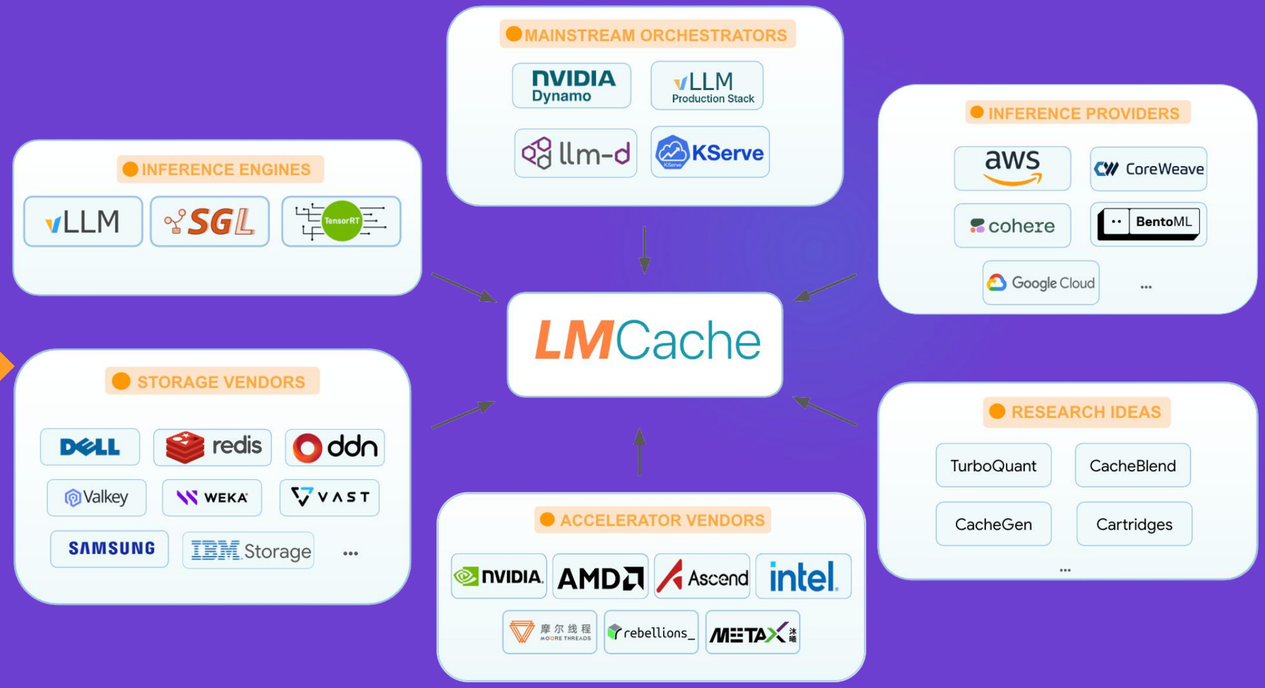
统一缓存层把两两适配收敛为标准边界

没有统一层：N × M 两两适配



每增加一个引擎或后端，都可能引入一批新组合

引入 LMCache：引擎和后端各接一次



统一边界减少组合爆炸，让引擎、存储和硬件按各自节奏演进

从一个可验证的小贡献开始

01

First-good-issue

新手村任务

02

Small PR

小 bugfix，UT 修复，CI 改进，
可观测提升

03

Medium PR

小特性，小性能优化

04

Features

大的 feature
重要的性能优化
架构重构

文档 · 示例 · 测试 · 平台适配 · L2 Adapter

github.com/LMCache/LMCache → Issues / Contributing

04 LMCache 部署、可观测性和容错

生成环境部署

01 声明配置

LMCacheEngine

自定义资源 CR

容量	缓存空间规划
端口	服务连接配置
L2	存储后端配置
Prometheus	监控接入配置

02 持续收敛

LMCache Operator



读取 CR 中的期望状态
协调部署对象
持续检查与更新

03 部署输出

DaemonSet + Service

每节点部署 LMCache Server

ConfigMap + Monitor

配置管理与 Prometheus 监控

Webhook

可选注入 vLLM 连接

一个 CR 描述期望状态，Operator 负责让部署对象持续收敛

生成环境部署

01 声明配置

LMCacheEngine

自定义资源 CR

容量	缓存空间规划
端口	服务连接配置
L2	存储后端配置
Prometheus	监控接入配置

02 持续收敛

LMCache Operator



读取 CR 中的期望状态
协调部署对象
持续检查与更新

03 部署输出

DaemonSet + Service

每节点部署 LMCache Server

ConfigMap + Monitor

配置管理与 Prometheus 监控

Webhook

可选注入 vLLM 连接

一个 CR 描述期望状态，Operator 负责让部署对象持续收敛

指标、日志和 Tracing 各自回答不同问题

Metrics

看趋势

命中、容量和性能如何变化?

命中率 / 容量
吞吐 / 延迟

时间序列指标，观察长期趋势

Logging

看状态

缓存正在执行什么操作?

DEBUG 日志
L1 / L2 / prefetch

运行日志，解释状态与异常

Tracing

看链路

请求耗时集中在哪个阶段?

OTel spans
阶段耗时 / 链路瓶颈

追踪请求链路，定位耗时阶段

联合诊断 **指标识别趋势 → 日志解释状态 → Trace 定位链路**

Prometheus 负责长期趋势，HTTP / CLI 负责即时诊断

指标、日志和 Tracing 各自回答不同问题

Metrics

看趋势

命中、容量和性能如何变化?

命中率 / 容量
吞吐 / 延迟

时间序列指标，观察长期趋势

Logging

看状态

缓存正在执行什么操作?

DEBUG 日志
L1 / L2 / prefetch

运行日志，解释状态与异常

Tracing

看链路

请求耗时集中在哪个阶段?

OTel spans
阶段耗时 / 链路瓶颈

追踪请求链路，定位耗时阶段

联合诊断 **指标识别趋势 → 日志解释状态 → Trace 定位链路**

Prometheus 负责长期趋势，HTTP / CLI 负责即时诊断

生产可观测性支持

生产洞察

全时健康监控

实时追踪节点状态与核心指标。

多层级容量透视

可视化 GPU / DRAM 分层水位。

读写异常根因定位

精准捕获失败请求的错误码，快速排查链路故障点。

Serving 引擎拓扑

展示在线引擎节点、版本分布及负载热力图。

性能调优

缓存命中率分析

监控 KV Cache 命中趋势，量化利用率。

全生命周期追踪

记录 KV 从创建、命中到淘汰的全链路轨迹。

冷热数据周转分析

统计淘汰至复用的时间差，评估淘汰策略合理性。

IOPS 与延迟透视

监测缓存读写吞吐与响应延迟，定位存储 I/O 瓶颈。

标准化可观测性底座：Prometheus & OpenTelemetry

统一采集、存储与关联分析 Metrics、Logs、Traces，为高并发生产环境提供高可用、高扩展的监控基础设施。

生产可观测性支持

生产洞察

全时健康监控

实时追踪节点状态与核心指标。

多层级容量透视

可视化 GPU / DRAM 分层水位。

读写异常根因定位

精准捕获失败请求的错误码，快速排查链路故障点。

Serving 引擎拓扑

展示在线引擎节点、版本分布及负载热力图。

性能调优

缓存命中率分析

监控 KV Cache 命中趋势，量化利用率。

全生命周期追踪

记录 KV 从创建、命中到淘汰的全链路轨迹。

冷热数据周转分析

统计淘汰至复用的时间差，评估淘汰策略合理性。

IOPS 与延迟透视

监测缓存读写吞吐与响应延迟，定位存储 I/O 瓶颈。

标准化可观测性底座：Prometheus & OpenTelemetry

统一采集、存储与关联分析 Metrics、Logs、Traces，为高并发生产环境提供高可用、高扩展的监控基础设施。

排障：沿四层证据链定位问题

01	服务与连接	检查	ping / health / host:port
		判断	服务可达，地址与端口匹配。
02	请求是否进入	检查	lookup_requested 是否增长
		判断	若不增长，先查连接器与请求路由。
03	L1 / L2 数据流	检查	store → prefetch → load
		判断	定位卡在写入、预取还是加载。
04	线程与完整性	检查	threads / checksums / trace
		判断	排查后台线程阻塞与数据损坏。

诊断工具箱

路径与趋势

GET /metrics

动态调试

GET /loglevel · GET /threads

周期任务

GET /periodic-threads

完整性校验

POST /cache/checksums

先用计数器确认路径，再用 DEBUG 日志解释原因

排障：沿四层证据链定位问题

01	服务与连接	检查	ping / health / host:port
		判断	服务可达，地址与端口匹配。
02	请求是否进入	检查	lookup_requested 是否增长
		判断	若不增长，先查连接器与请求路由。
03	L1 / L2 数据流	检查	store → prefetch → load
		判断	定位卡在写入、预取还是加载。
04	线程与完整性	检查	threads / checksums / trace
		判断	排查后台线程阻塞与数据损坏。

诊断工具箱

路径与趋势

GET /metrics

动态调试

GET /loglevel · GET /threads

周期任务

GET /periodic-threads

完整性校验

POST /cache/checksums

先用计数器确认路径，再用 DEBUG 日志解释原因

生产环境中的故障容错

01 vLLM 主进程意外崩溃

vLLM 进程

异常退出 / 请求中断

LMCache 进程

独立运行 / 缓存保留

MP Server 独立维护 CPU 上的 KV Cache。vLLM 退出后，缓存继续驻留；引擎重启后可以直接复用，减少冷启动重建开销。

恢复路径：进程重启 → 连接 MP Server → 复用缓存

02 LMCache 缓存服务宕机

vLLM 进程

自动降级 / 推理继续

LMCache 进程

连接超时 / 服务不可用

推理引擎回退到本地 KV Cache，维持服务。LMCache 恢复后，通过心跳检测自动重连，再切回共享缓存模式。

恢复路径：本地缓存降级 → 心跳重连 → 共享缓存

进程隔离 + 自动降级：让缓存层故障具备可预测的恢复路径

生产环境中的故障容错

01 vLLM 主进程意外崩溃

vLLM 进程

异常退出 / 请求中断

LMCache 进程

独立运行 / 缓存保留

MP Server 独立维护 CPU 上的 KV Cache。vLLM 退出后，缓存继续驻留；引擎重启后可以直接复用，减少冷启动重建开销。

恢复路径：进程重启 → 连接 MP Server → 复用缓存

02 LMCache 缓存服务宕机

vLLM 进程

自动降级 / 推理继续

LMCache 进程

连接超时 / 服务不可用

推理引擎回退到本地 KV Cache，维持服务。LMCache 恢复后，通过心跳检测自动重连，再切回共享缓存模式。

恢复路径：本地缓存降级 → 心跳重连 → 共享缓存

进程隔离 + 自动降级：让缓存层故障具备可预测的恢复路径

05 LMCache 使用场景与最佳实践

场景一：KV Cache Offload：冷数据下沉与按需回迁

GPU

最热 KV Cache

访问特征
最低延迟，容量最贵
驻留内容
当前计算所需的热数据
管理重点
为活跃请求保留空间

evict 下沉 →

← load 回迁

CPU L1

节点内共享热集

访问特征
较大容量，节点内复用
驻留内容
从 GPU 下沉的可复用 KV
管理重点
连接 GPU 与 L2 的缓存层

store 写入 →

← prefetch 预取

L2

远端或本地存储

访问特征
更低层存储，回迁延迟更高
驻留内容
下沉保存的 KV Cache
管理重点
关注存储吞吐与读取延迟

数据流　　冷数据逐层下沉，命中后再加载到计算所需的层级。

Offload 收益判断：节省的 Prefill 时间 – 数据回迁时间

结合真实请求测量命中情况与回迁开销，再决定容量和下沉策略。

场景一：KV Cache Offload：冷数据下沉与按需回迁

GPU

最热 KV Cache

访问特征
最低延迟，容量最贵
驻留内容
当前计算所需的热数据
管理重点
为活跃请求保留空间

evict 下沉 →

← load 回迁

CPU L1

节点内共享热集

访问特征
较大容量，节点内复用
驻留内容
从 GPU 下沉的可复用 KV
管理重点
连接 GPU 与 L2 的缓存层

store 写入 →

← prefetch 预取

L2

远端或本地存储

访问特征
更低层存储，回迁延迟更高
驻留内容
下沉保存的 KV Cache
管理重点
关注存储吞吐与读取延迟

数据流　　冷数据逐层下沉，命中后再加载到计算所需的层级。

Offload 收益判断：节省的 Prefill 时间 – 数据回迁时间

结合真实请求测量命中情况与回迁开销，再决定容量和下沉策略。

场景二：P/D 分离：P2P 与组合连接两条路径

路径 A：LMCache P2P

直接完成 P → D 的 KV 传输

Prefill

LMCacheMPConnector

NIXL

Decode

同一请求继续 decode

公开示例覆盖范围

单节点、2 GPU、直接 P → D 传输

路径 B：MultiConnector 组合

组合 P/D 传输与 Offload 能力

P/D Connector

负责 P/D 传输

+

Offload

缓存下沉与持久化

组合接入验证

固定 vLLM / LMCache 版本，
验证双连接器完成事件与失败回退。

01 **是否同时 Offload**

先明确传输与缓存需求

02 **版本兼容性**

固定并验证版本组合

03 **失败与回退**

检查完成事件和回退行为

场景二：P/D 分离：P2P 与组合连接两条路径

路径 A：LMCache P2P

直接完成 P → D 的 KV 传输

Prefill

LMCacheMPConnector

NIXL

Decode

同一请求继续 decode

公开示例覆盖范围

单节点、2 GPU、直接 P → D 传输

路径 B：MultiConnector 组合

组合 P/D 传输与 Offload 能力

P/D Connector

负责 P/D 传输

+

Offload

缓存下沉与持久化

组合接入验证

固定 vLLM / LMCache 版本，
验证双连接器完成事件与失败回退。

01 **是否同时 Offload**

先明确传输与缓存需求

02 **版本兼容性**

固定并验证版本组合

03 **失败与回退**

检查完成事件和回退行为

最佳实践一：容量与命中率：从工作集开始估算

01 每 token KV 字节

2 × layers × KV heads × head_dim × bytes

2 对应 K 与 V，bytes 为每个元素的字节数。

02 可容纳 token

可用缓存字节 ÷ 每 token KV 字节

将缓存空间换算成可驻留的 token 数量。

03 可复用 token

floor(shared prefix ÷ chunk) × chunk

共享前缀按完整 chunk 向下对齐。

04 估算有效命中率

可复用 token × 驻留概率 ÷ lookup token

同时考虑前缀复用与数据仍在缓存中的概率。

先分析 trace 中的共享前缀与工作集，再决定容量

最佳实践一：容量与命中率：从工作集开始估算

01 每 token KV 字节

2 × layers × KV heads × head_dim × bytes

2 对应 K 与 V，bytes 为每个元素的字节数。

02 可容纳 token

可用缓存字节 ÷ 每 token KV 字节

将缓存空间换算成可驻留的 token 数量。

03 可复用 token

floor(shared prefix ÷ chunk) × chunk

共享前缀按完整 chunk 向下对齐。

04 估算有效命中率

可复用 token × 驻留概率 ÷ lookup token

同时考虑前缀复用与数据仍在缓存中的概率。

先分析 trace 中的共享前缀与工作集，再决定容量

最佳实践一：如何用lmcache CLI估算缓存容量

01 **采集真实工作集**

`lmcache server --lookup-hash-log-dir ...`

记录 chunk hash、序列长度、shape 与 dtype

02 **按 LRU 回放**

`cache-simulator simulate`

自动计算每个 chunk 的字节数，复现前缀命中

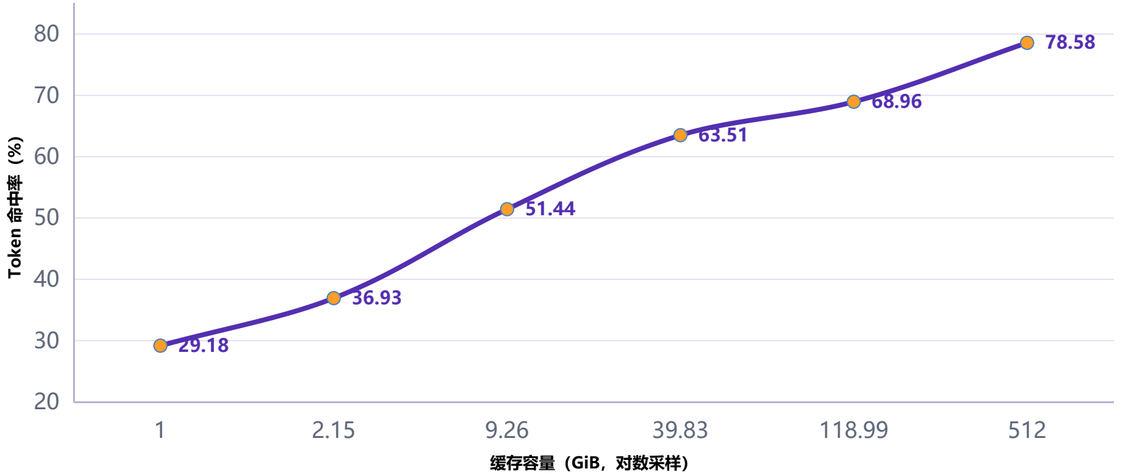
03 **扫描容量区间**

`cache-simulator sweep`

得到容量与 token 命中率曲线，选择目标点

示例：容量扫描曲线

横轴为缓存容量（GiB），纵轴为 token 命中率



容量选择：按目标命中率或曲线拐点取值，并预留运行余量

用真实请求分布做容量扫描，比仅按模型参数估算更接近线上命中率

最佳实践一：如何用lmcache CLI估算缓存容量

01 **采集真实工作集**

`lmcache server --lookup-hash-log-dir ...`

记录 chunk hash、序列长度、shape 与 dtype

02 **按 LRU 回放**

`cache-simulator simulate`

自动计算每个 chunk 的字节数，复现前缀命中

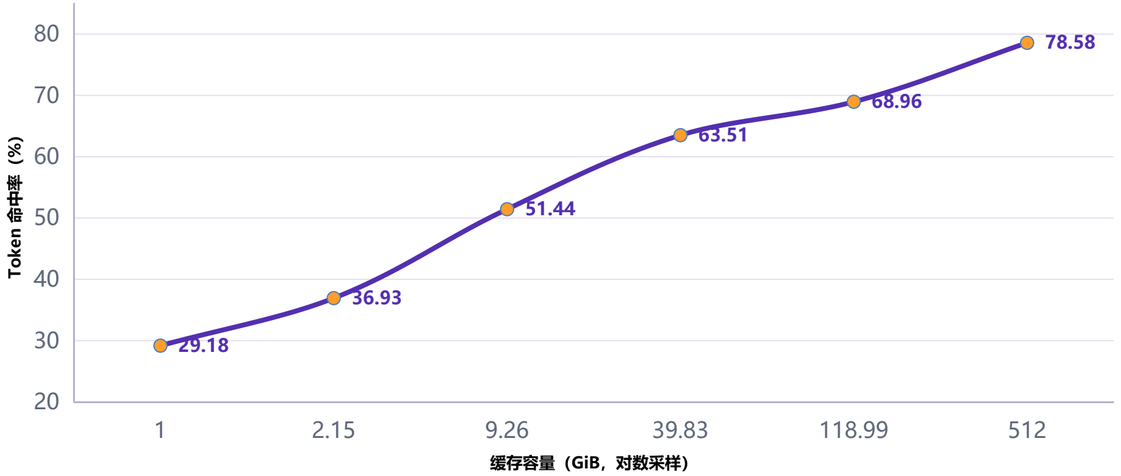
03 **扫描容量区间**

`cache-simulator sweep`

得到容量与 token 命中率曲线，选择目标点

示例：容量扫描曲线

横轴为缓存容量（GiB），纵轴为 token 命中率



容量选择：按目标命中率或曲线拐点取值，并预留运行余量

用真实请求分布做容量扫描，比仅按模型参数估算更接近线上命中率

最佳实践一：如何用lmcache CLI估算缓存容量

01 **采集真实工作集**

`lmcache server --lookup-hash-log-dir ...`

记录 chunk hash、序列长度、shape 与 dtype

02 **按 LRU 回放**

`cache-simulator simulate`

自动计算每个 chunk 的字节数，复现前缀命中

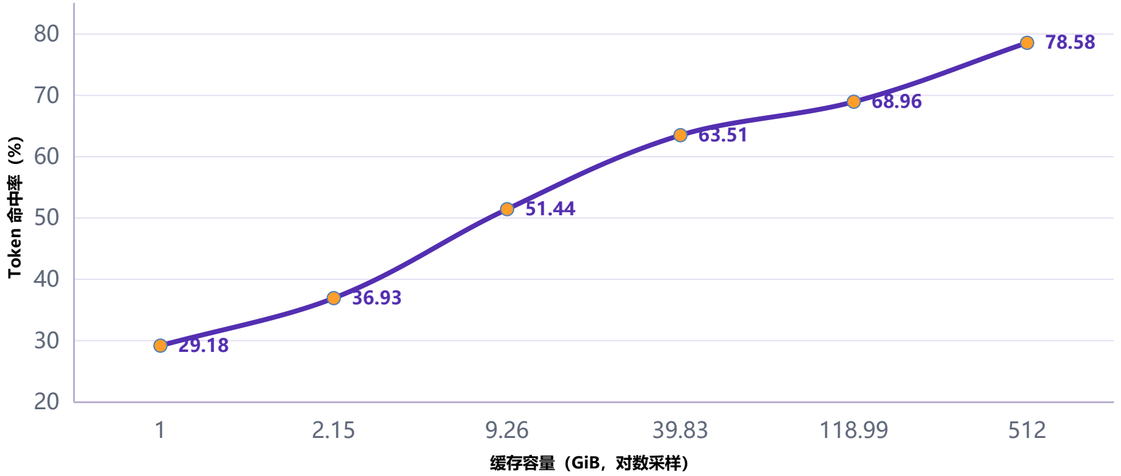
03 **扫描容量区间**

`cache-simulator sweep`

得到容量与 token 命中率曲线，选择目标点

示例：容量扫描曲线

横轴为缓存容量（GiB），纵轴为 token 命中率



容量选择：按目标命中率或曲线拐点取值，并预留运行余量

用真实请求分布做容量扫描，比仅按模型参数估算更接近线上命中率

最佳实践二：Chunk Size：在粒度与开销之间取平衡

同样的 1024 tokens，按不同 chunk size 切分

128
tokens / chunk



8 个完整 chunk

命中粒度更细

块数更多，操作开销更高

256
tokens / chunk



4 个完整 chunk

作为调优起点

在命中粒度与操作开销间取平衡

512
tokens / chunk



2 个完整 chunk

大块传输更利于吞吐

尾部不足一块时，复用损失更大

从 256 开始，用 128 / 256 / 512 对真实流量做 A/B

比较命中率、吞吐与延迟，结合真实请求的前缀分布选择。

最佳实践二：Chunk Size：在粒度与开销之间取平衡

同样的 1024 tokens，按不同 chunk size 切分

128
tokens / chunk



8 个完整 chunk

命中粒度更细

块数更多，操作开销更高

256
tokens / chunk



4 个完整 chunk

作为调优起点

在命中粒度与操作开销间取平衡

512
tokens / chunk



2 个完整 chunk

大块传输更利于吞吐

尾部不足一块时，复用损失更大

从 256 开始，用 128 / 256 / 512 对真实流量做 A/B

比较命中率、吞吐与延迟，结合真实请求的前缀分布选择。

最佳实践三：本地轻量验证

01

CPU / CLI / L2

先确认基础链路

验证目标	连接、配置、指标与 I/O
执行方式	检查服务连接和配置生效， 执行 L2 读写并观察指标。
检查结果	服务可达，读写可完成， 相关计数器随操作变化。

02

hf_overrides + dummy

再验证启动与数据路径

验证目标	引擎能启动，缓存路径能走通
执行方式	使用轻量配置启动， 发起请求并检查日志、指标。
检查结果	确认请求进入缓存路径， 观察 store / prefetch / load。

03

小型真实模型

最后验证命中与输出

验证目标	重复请求命中与模型输出
执行方式	运行真实小模型，重复发送 相同前缀请求，对比输出。
检查结果	结合命中指标与输出对比， 验证缓存复用行为。

验证顺序　　**基础服务与 I/O → 引擎启动与缓存路径 → 真实请求的命中与输出**

验证边界：dummy 证明路径可走通，不能证明真实模型的输出正确

最佳实践三：本地轻量验证

01

CPU / CLI / L2

先确认基础链路

验证目标	连接、配置、指标与 I/O
执行方式	检查服务连接和配置生效， 执行 L2 读写并观察指标。
检查结果	服务可达，读写可完成， 相关计数器随操作变化。

02

hf_overrides + dummy

再验证启动与数据路径

验证目标	引擎能启动，缓存路径能走通
执行方式	使用轻量配置启动， 发起请求并检查日志、指标。
检查结果	确认请求进入缓存路径， 观察 store / prefetch / load。

03

小型真实模型

最后验证命中与输出

验证目标	重复请求命中与模型输出
执行方式	运行真实小模型，重复发送 相同前缀请求，对比输出。
检查结果	结合命中指标与输出对比， 验证缓存复用行为。

验证顺序　　**基础服务与 I/O → 引擎启动与缓存路径 → 真实请求的命中与输出**

验证边界：dummy 证明路径可走通，不能证明真实模型的输出正确

最佳实践三：本地轻量验证

01

CPU / CLI / L2

先确认基础链路

验证目标	连接、配置、指标与 I/O
执行方式	检查服务连接和配置生效， 执行 L2 读写并观察指标。
检查结果	服务可达，读写可完成， 相关计数器随操作变化。

02

hf_overrides + dummy

再验证启动与数据路径

验证目标	引擎能启动，缓存路径能走通
执行方式	使用轻量配置启动， 发起请求并检查日志、指标。
检查结果	确认请求进入缓存路径， 观察 store / prefetch / load。

03

小型真实模型

最后验证命中与输出

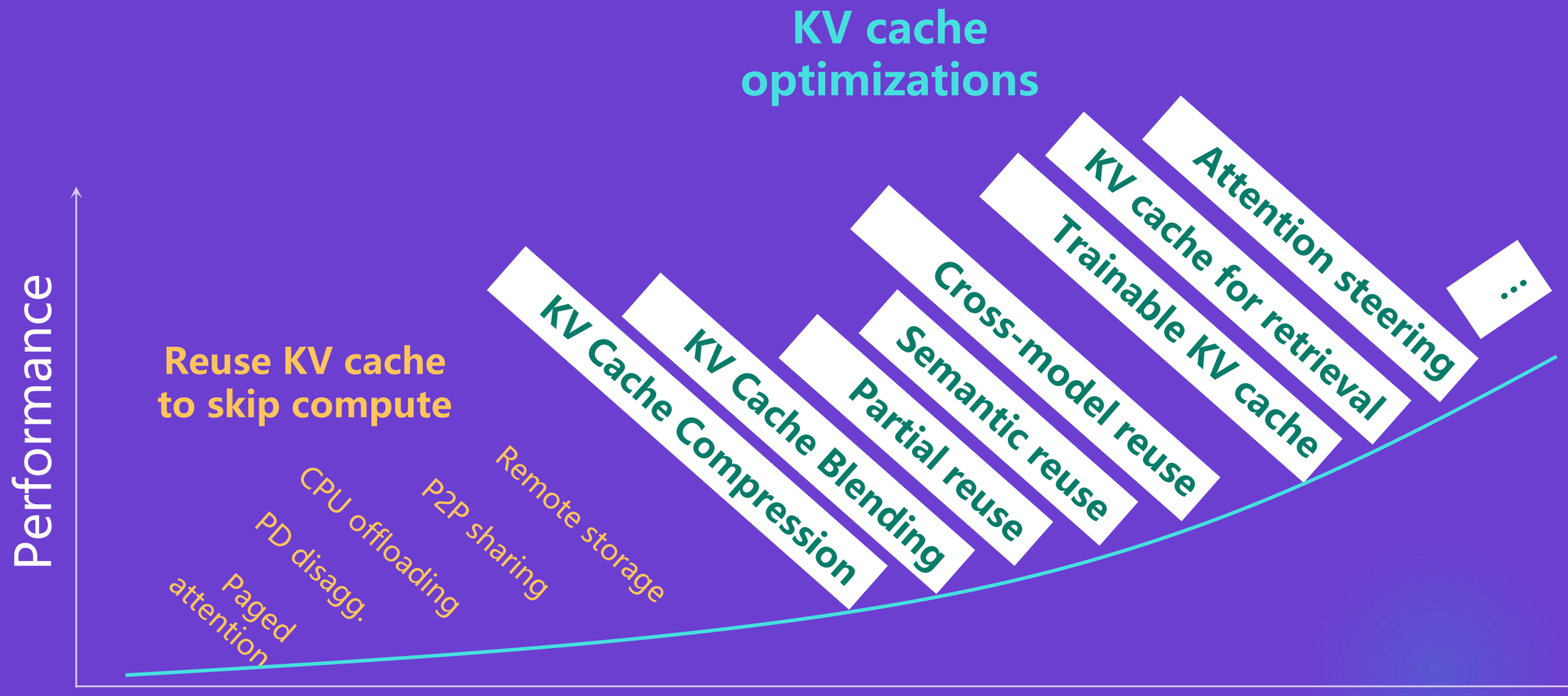
验证目标	重复请求命中与模型输出
执行方式	运行真实小模型，重复发送 相同前缀请求，对比输出。
检查结果	结合命中指标与输出对比， 验证缓存复用行为。

验证顺序　　**基础服务与 I/O → 引擎启动与缓存路径 → 真实请求的命中与输出**

验证边界：dummy 证明路径可走通，不能证明真实模型的输出正确

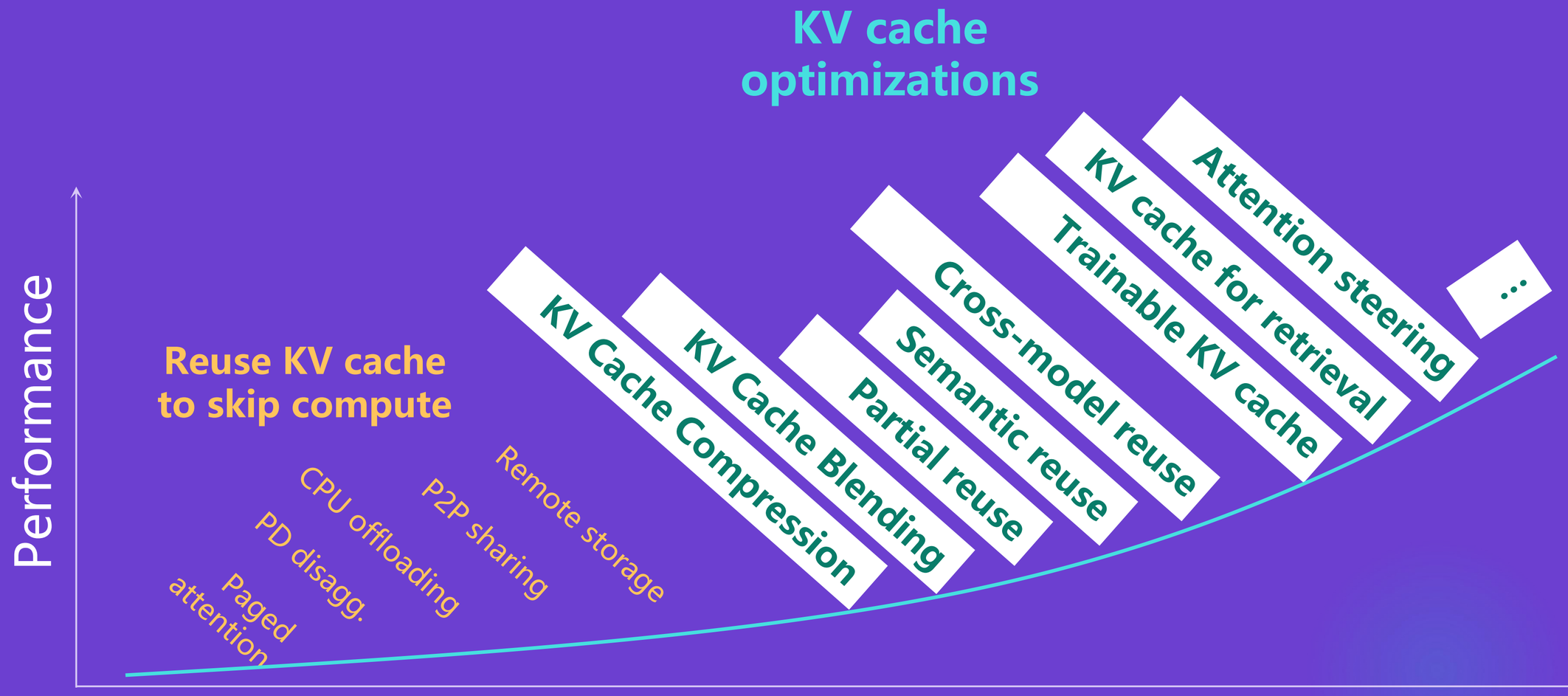
06 LMCache 展望和Roadmap

LMCache 未来展望



LMCache 致力于将 KV Cache 从单机状态升级为可跨实例、跨节点复用的分布式缓存基础设施。

LMCache 未来展望



LMCache 致力于将 KV Cache 从单机状态升级为可跨实例、跨节点复用的分布式缓存基础设施。

未来 Roadmap

01 持续优化



持续完善核心功能
优化缓存效率与端到端性能

02 模型支持



持续适配最新模型
扩展不同架构与多模态模型支持

03 推理引擎支持



接入更多推理引擎
持续跟进现有引擎的新功能

04 生产落地



完善部署运维与可观测性
提升复杂生产场景下的稳定性

未来 Roadmap

01 持续优化



持续完善核心功能
优化缓存效率与端到端性能

02 模型支持



持续适配最新模型
扩展不同架构与多模态模型支持

03 推理引擎支持



接入更多推理引擎
持续跟进现有引擎的新功能

04 生产落地



完善部署运维与可观测性
提升复杂生产场景下的稳定性

欢迎加入 LMCache 社区继续共建

Slack

扫码加入 LMCache Slack
#lmcache

LMCache

Q & A

GitHub
github.com/LMCache/LMCache

Docs
docs.lmcache.ai

一起讨论，一起提交，一起验证

Github: LMCache

扫码跳转 github 地址

欢迎加入 LMCache 社区继续共建

Slack

扫码加入 LMCache Slack
#lmcache

LMCache

Q & A

GitHub
github.com/LMCache/LMCache

Docs
docs.lmcache.ai

一起讨论，一起提交，一起验证

Github: LMCache

扫码跳转 github 地址